

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа

«Внешние сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

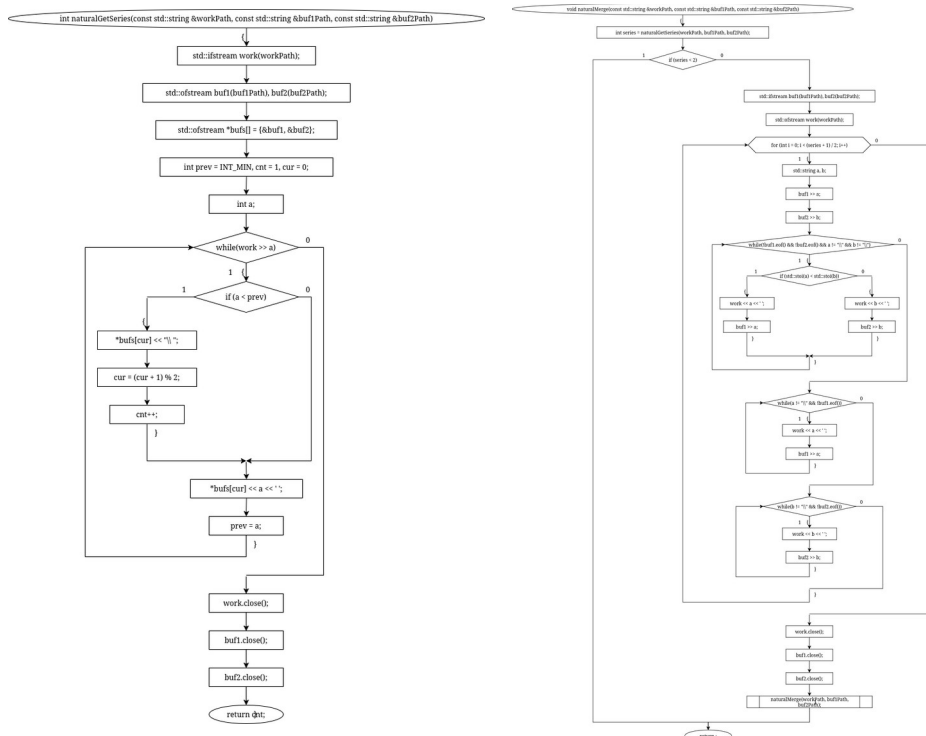
Реализовать алгоритмы внешних сортировок:

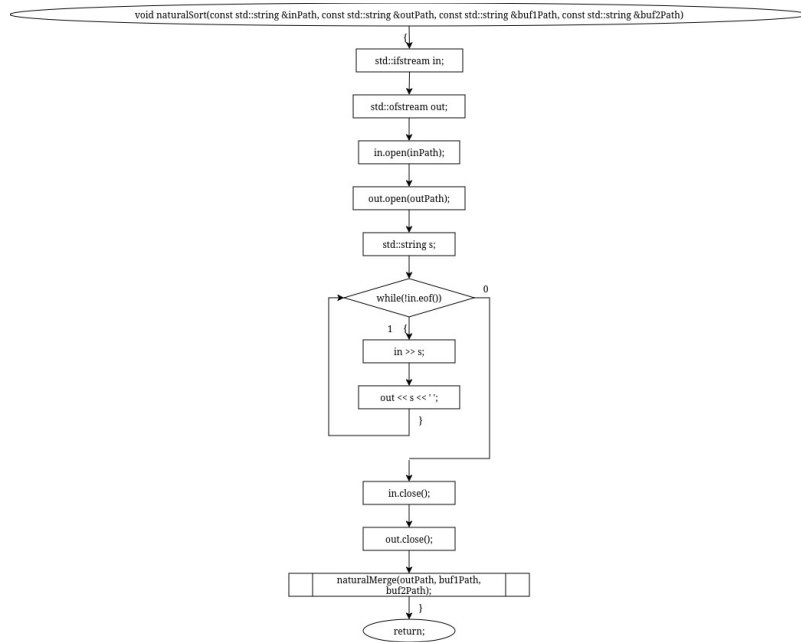
1. Метод естественного слияния.
2. Сбалансированное двухфазное слияние.
3. Многофазная сортировка.

АНАЛИЗ МЕТОДА ЕСТЕСТВЕННОГО СЛИЯНИЯ

1. Идём по файлу и делим на подряд идущие неубывающие куски.
2. Распределяем серии: по очереди записываем серии в два буфера, разделяя их маркером \.
3. Сливаем серии попарно: берём по одной серии из каждого буфера и сливаем, как в обычном merge, пока не упрёмся в \.
4. Повторяем: снова ищем серии уже в “рабочем” файле и сливаем, пока не останется одна серия.

БЛОК-СХЕМА





КОД

```

#include <fstream>
#include <climits>
#include <string>

```

```

int naturalGetSeries(const std::string &workPath, const std::string &buf1Path, const std::string
&buf2Path) {
    std::ifstream work(workPath);
    std::ofstream buf1(buf1Path), buf2(buf2Path);
    std::ofstream *bufs[] = {&buf1, &buf2};
    int prev = INT_MIN, cnt = 1, cur = 0;
    int a;
    while (work >> a) {
        if (a < prev) {
            *bufs[cur] << "\\ ";
            cur = (cur + 1) % 2;
            cnt++;
        }
        *bufs[cur] << a << ' ';
        prev = a;
    }
    work.close();
    buf1.close();
    buf2.close();
    return cnt;
}

```

```

void naturalMerge(const std::string &workPath, const std::string &buf1Path,
                 const std::string &buf2Path) {
    int series = naturalGetSeries(workPath, buf1Path, buf2Path);
    if (series < 2)
        return;
    std::ifstream buf1(buf1Path), buf2(buf2Path);
    std::ofstream work(workPath);

    for (int i = 0; i < (series + 1) / 2; i++) {
        std::string a, b;
        buf1 >> a;
        buf2 >> b;

        while (!buf1.eof() && !buf2.eof() && a != "\\\" && b != "\\\"") {
            if (std::stoi(a) < std::stoi(b)) {
                work << a << ' ';
                buf1 >> a;
            } else {
                work << b << ' ';
                buf2 >> b;
            }
        }

        while (a != "\\\" && !buf1.eof()) {
            work << a << ' ';
            buf1 >> a;
        }

        while (b != "\\\" && !buf2.eof()) {
            work << b << ' ';
            buf2 >> b;
        }
    }
    work.close();
    buf1.close();
    buf2.close();
    naturalMerge(workPath, buf1Path, buf2Path);
}

```

```

void naturalSort(const std::string &inPath, const std::string &outPath, const std::string
&buf1Path,
                const std::string &buf2Path) {
    std::ifstream in;
    std::ofstream out;
    in.open(inPath);

```

```

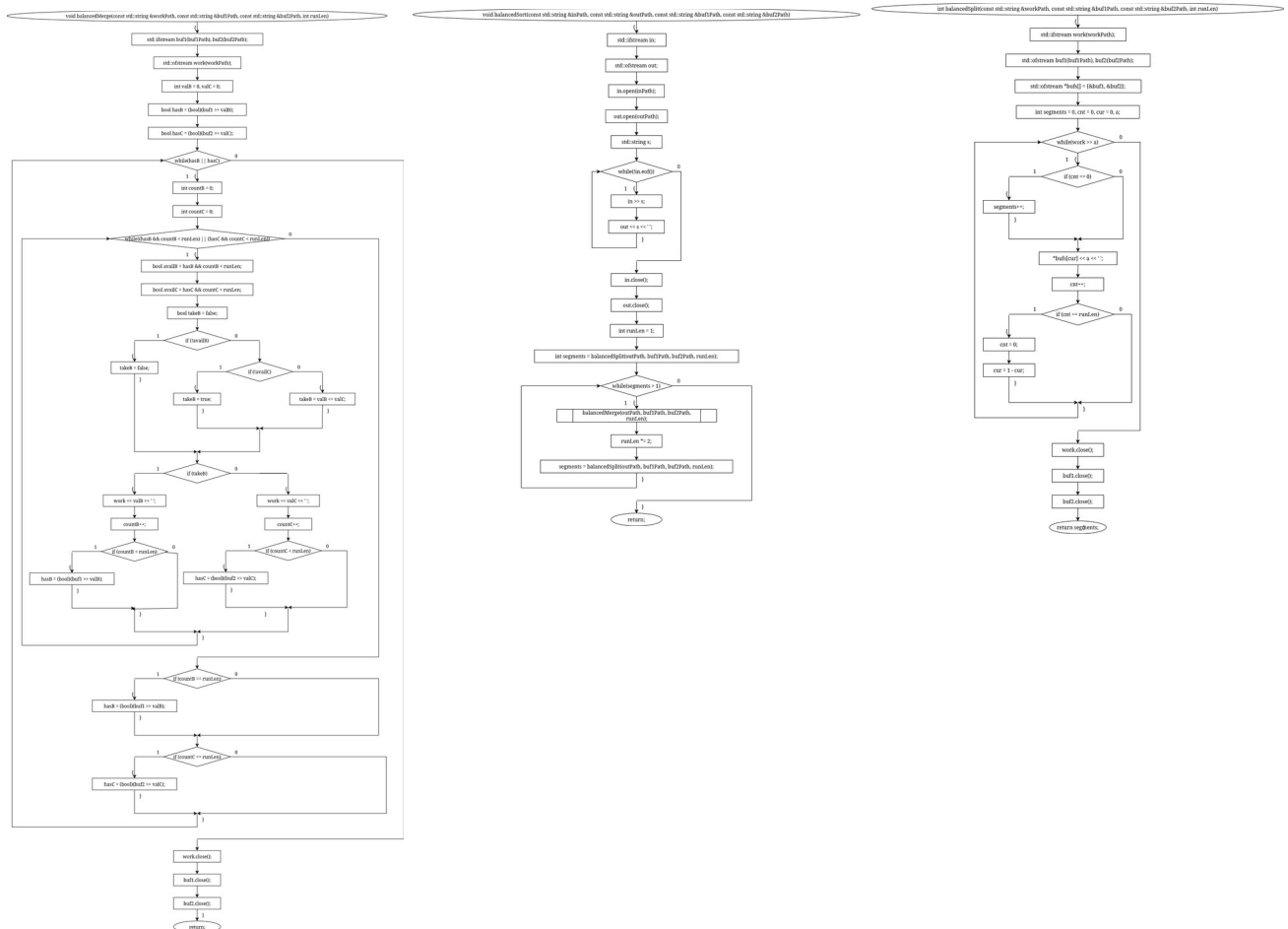
out.open(outPath);
std::string s;
while (!in.eof()) {
    in >> s;
    out << s << ' ';
}
in.close();
out.close();
naturalMerge(outPath, buf1Path, buf2Path);
}

```

АНАЛИЗ МЕТОДА ДВУХФАЗНОГО СЛИЯНИЯ

1. В отличие от первого метода, не ищем естественные куски, а режем файл на серии длины 1, потом 2, 4, 8 и так далее.
2. Далее разделяем по сериям. Первые runLen элементов — в первый буфер, следующие runLen — во второй, и так далее.
3. Берём две серии одинаковой длины и сливаем их в основной файл.
4. Удваиваем длину: $\text{runLen} *= 2$ и повторяем, пока весь файл не станет одной серией.

БЛОК-СХЕМА



Код

```
#include <fstream>
#include <string>
```

```
int balancedSplit(const std::string &workPath, const std::string &buf1Path, const std::string
&buf2Path,
```

```
    int runLen) {
    std::ifstream work(workPath);
    std::ofstream buf1(buf1Path), buf2(buf2Path);
    std::ofstream *bufs[] = {&buf1, &buf2};

    int segments = 0, cnt = 0, cur = 0, a;
    while (work >> a) {
        if (cnt == 0) {
            segments++;
        }
        *bufs[cur] << a << ' ';
        cnt++;
        if (cnt == runLen) {
            cnt = 0;
            cur = 1 - cur;
        }
    }
    work.close();
    buf1.close();
    buf2.close();
    return segments;
}
```

```
void balancedMerge(const std::string &workPath, const std::string &buf1Path, const std::string
&buf2Path,
```

```
    int runLen) {
    std::ifstream buf1(buf1Path), buf2(buf2Path);
    std::ofstream work(workPath);

    int valB = 0, valC = 0;
    bool hasB = (bool)(buf1 >> valB);
    bool hasC = (bool)(buf2 >> valC);

    while (hasB || hasC) {
        int countB = 0;
        int countC = 0;

        while ((hasB && countB < runLen) || (hasC && countC < runLen)) {
```

```

    bool availB = hasB && countB < runLen;
    bool availC = hasC && countC < runLen;
    bool takeB = false;

    if (!availB) {
        takeB = false;
    } else if (!availC) {
        takeB = true;
    } else {
        takeB = valB <= valC;
    }

    if (takeB) {
        work << valB << ' ';
        countB++;
        if (countB < runLen) {
            hasB = (bool)(buf1 >> valB);
        }
    } else {
        work << valC << ' ';
        countC++;
        if (countC < runLen) {
            hasC = (bool)(buf2 >> valC);
        }
    }
}

if (countB == runLen) {
    hasB = (bool)(buf1 >> valB);
}
if (countC == runLen) {
    hasC = (bool)(buf2 >> valC);
}
}

work.close();
buf1.close();
buf2.close();
}

void balancedSort(const std::string &inPath, const std::string &outPath, const std::string
&buf1Path,
                 const std::string &buf2Path) {
    std::ifstream in;
    std::ofstream out;

```

```

in.open(inPath);
out.open(outPath);
std::string s;
while (!in.eof()) {
    in >> s;
    out << s << '\n';
}
in.close();
out.close();

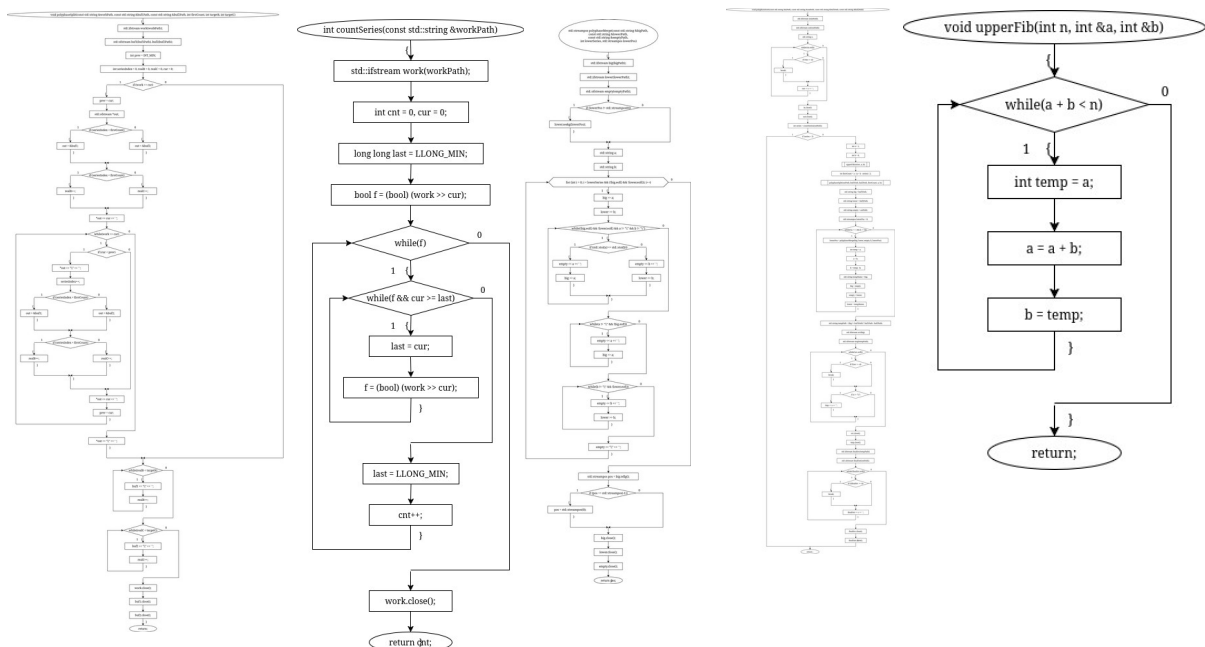
int runLen = 1;
int segments = balancedSplit(outPath, buf1Path, buf2Path, runLen);
while (segments > 1) {
    balancedMerge(outPath, buf1Path, buf2Path, runLen);
    runLen *= 2;
    segments = balancedSplit(outPath, buf1Path, buf2Path, runLen);
}
}

```

АНАЛИЗ МЕТОДА МНОГОФАЗНОГО ДЕЛЕНИЯ

1. Считаем число естественных серий во входе.
2. Подбираем два числа Фибоначчи таких, что их сумма покрывает число серий. Недостающее добиваем фиктивными сериями ().
3. Начальное распределение осуществляем следующим образом: первые $F(k)$ серий — в один файл, $F(k-1)$ — в другой. Третий файл пустой.
4. Сливаем серии из двух входных в пустой, пока один входной не исчерпается.
5. Далее пустой файл становится входным, один из входных — пустым. Счётчики серий обновляются по Фибоначчи.
6. Повторяем, пока не останется одна серия — это и есть отсортированный файл.

БЛОК-СХЕМА



КОД

```
#include <fstream>
#include <string>
#include <climits>

void upperFib(int n, int &a, int &b) {
    while (a + b < n) {
        int temp = a;
        a = a + b;
        b = temp;
    }
}

int countSeries(const std::string &workPath) {
    std::ifstream work(workPath);
    int cnt = 0, cur = 0;

    long long last = LLONG_MIN;
    bool f = (bool) (work >> cur);
    while (f) {
        while (f && cur >= last) {
            last = cur;
            f = (bool) (work >> cur);
        }
        last = LLONG_MIN;
        cnt++;
    }
    work.close();
    return cnt;
}

void polyphaseSplit(const std::string &workPath, const std::string &buf1Path, const std::string
&buf2Path,
                    int firstCount, int targetB, int targetC) {
    std::ifstream work(workPath);
    std::ofstream buf1(buf1Path), buf2(buf2Path);

    int prev = INT_MIN;
    int seriesIndex = 0, realB = 0, realC = 0, cur = 0;
    if (work >> cur) {
        prev = cur;
        std::ofstream *out;
        if (seriesIndex < firstCount) out = &buf1;
        else out = &buf2;
```

```

    if (seriesIndex < firstCount) realB++;
    else realC++;

    *out << cur << ' ';
    while (work >> cur) {
        if (cur < prev) {
            *out << "\\ " << ' ';
            seriesIndex++;
            if (seriesIndex < firstCount) out = &buf1;
            else out = &buf2;

            if (seriesIndex < firstCount) realB++;
            else realC++;
        }
        *out << cur << ' ';
        prev = cur;
    }
    *out << "\\ " << ' ';
}

while (realB < targetB) {
    buf1 << "\\ " << ' ';
    realB++;
}
while (realC < targetC) {
    buf2 << "\\ " << ' ';
    realC++;
}

work.close();
buf1.close();
buf2.close();
}

std::streampos polyphaseMerge(const std::string &bigPath, const std::string &lowerPath,
                              const std::string &emptyPath, int lowerSeries,
                              std::streampos lowerPos) {
    std::ifstream big(bigPath);
    std::ifstream lower(lowerPath);
    std::ofstream empty(emptyPath);

    if (lowerPos != std::streampos(0)) lower.seekg(lowerPos);

```

```

std::string a;
std::string b;

for (int i = 0; i < lowerSeries && (!big.eof() && !lower.eof()); i++) {
    big >> a;
    lower >> b;

    while (!big.eof() && !lower.eof() && a != "\\" && b != "\\") {
        if (std::stoi(a) <= std::stoi(b)) {
            empty << a << ' ';
            big >> a;
        } else {
            empty << b << ' ';
            lower >> b;
        }
    }

    while (a != "\\" && !big.eof()) {
        empty << a << ' ';
        big >> a;
    }

    while (b != "\\" && !lower.eof()) {
        empty << b << ' ';
        lower >> b;
    }

    empty << "\\" << ' ';
}

std::streampos pos = big.tellg();
if (pos == std::streampos(-1)) pos = std::streampos(0);

big.close();
lower.close();
empty.close();
return pos;
}

void polyphaseSort(const std::string &inPath, const std::string &outPath, const std::string
&buf1Path,
    const std::string &buf2Path) {
    std::ifstream in(inPath);
    std::ofstream out(outPath);
    std::string s;

```

```

while (!in.eof()) {
    if (!(in >> s)) break;
    out << s << ' ';
}
in.close();
out.close();

int series = countSeries(outPath);
if (series < 2) return;

int a = 1;
int b = 0;
upperFib(series, a, b);

int firstCount = a - (a + b - series) / 2;
polyphaseSplit(outPath, buf1Path, buf2Path, firstCount, a, b);

std::string big = buf1Path;
std::string lower = buf2Path;
std::string empty = outPath;
std::streampos lowerPos = 0;

while (!(a == 1 && b == 0)) {
    lowerPos = polyphaseMerge(big, lower, empty, b, lowerPos);

    int temp = a;
    a = b;
    b = temp - b;

    std::string tempName = big;
    big = empty;
    empty = lower;
    lower = tempName;
}

std::string tempPath = (big != buf1Path) ? buf1Path : buf2Path;
std::ifstream src(big);
std::ofstream tmp(tempPath);
while (!src.eof()) {
    if (!(src >> s)) break;
    if (s != "\\") tmp << s << ' ';
}
src.close();
tmp.close();

```

```

std::ifstream finalSrc(tempPath);
std::ofstream finalOut(outPath);
while (!finalSrc.eof()) {
    if (!(finalSrc >> s)) break;
    finalOut << s << ' ';
}
finalSrc.close();
finalOut.close();
}

```

БЛОК-СХЕМА ФУНКЦИИ int main()



КОД

```

#include <iostream>

#include "natural_merge_sort.cpp"
#include "balanced_merge_sort.cpp"
#include "polyphase_merge_sort.cpp"

using namespace std;

int main() {
    naturalSort("in", "naturalOut", "naturalBuf1", "naturalBuf2");
    balancedSort("in", "balancedOut", "balancedBuf1", "balancedBuf2");
    polyphaseSort("in", "polyphaseOut", "polyphaseBuf1", "polyphaseBuf2");
    ifstream in("in"), nout("naturalOut"), bout("balancedOut"), pout("polyphaseOut");
}

```

```

std::string s;
getline(in, s);
std::cout << s << "\n\n";

cout << "natural\n";
getline(nout, s);
cout << '\t' + s + '\n';

cout << "balanced\n";
getline(bout, s);
cout << '\t' + s + '\n';

cout << "polyphase\n";
getline(pout, s);
cout << '\t' + s + '\n';

}

```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

75 49 4 36 20 19 89 79 8 96 57 64 58 28 76 98 12 42 17 96

natural

4 8 12 17 19 20 28 36 42 49 57 58 64 75 76 79 89 96 96 98

balanced

4 8 12 17 19 20 28 36 42 49 57 58 64 75 76 79 89 96 96 98

polyphase

4 8 12 17 19 20 28 36 42 49 57 58 64 75 76 79 89 96 96 98